EXPERIMENT 08

User management and role based access

Aim

Apply least-privilege access for administrators, librarians, auditors and members, and verify that record ownership and sharing do not leak member data.

Procedure

1. Identify organization administrators separately from application users. Use distinct named test accounts. Do not give ordinary librarians organization administration or application-builder privileges.

2. Create custom permission sets and assign appropriate roles where supported. Roles describe hierarchy and sharing; permission sets control allowed operations. Review data-sharing rules as a separate layer.

3. Implement the intended matrix below. The administrator row describes a trusted application manager; organization-level operations require separately authorized administrators.

4. Start with access denied, then grant only the required forms, reports, fields and actions. Disable broad export, bulk delete and direct loan editing for circulation users unless there is an approved need.

Capability	App admin	Librarian	Auditor	Member
Catalogue and copies	Manage	Read and maintain	Read	Read catalogue
Member personal data	Manage	Read and maintain	Limited or redacted	Own only
Issue and return	Controlled action	Controlled action	No	No
Loan history	Read all	Read all	Read only	Own only
Fine correction	Audited approval	No direct edit	Read	Read own
Export records	Approved only	Off by default	Approved redacted	Off
Edit app or roles	Authorized admins	No	No	No
Delete loan history	Retention process	No	No	No

Member access design

If an external portal is available, define a portal permission set and map the authenticated portal identity to one Members record. Configure a supported server-enforced record-sharing or ownership mechanism so only that member’s loans are accessible. A loan entered by a librarian is normally created by the librarian, so “records added by the current user” is not automatically the member’s loan history.

Implementation and verification

Ifsecure memberscoping cannot be demonstrated intheavailable account, omit member access for the lab and mark it blocked. A report filter, hidden page element, or client-supplied member ID is not sufficient authorization.

Procedure continued

1. Review role hierarchy and data sharing for every form. Avoid enabling hierarchy sharing for members in a way that lets one member see another member’s records. Confirm any field-level restrictions also apply in reports and exports.

2. Where a custom function serves member data, derive the member from authenticated identity, not a request parameter. Reject zero or multiple identity matches. Consider the function’s execution identity: a privileged backend may need its own authorization checks.

3. Test direct record URLs, report URLs, exports, custom actions and APIs with separate browser sessions. Do not test every role while signed in as the application owner.

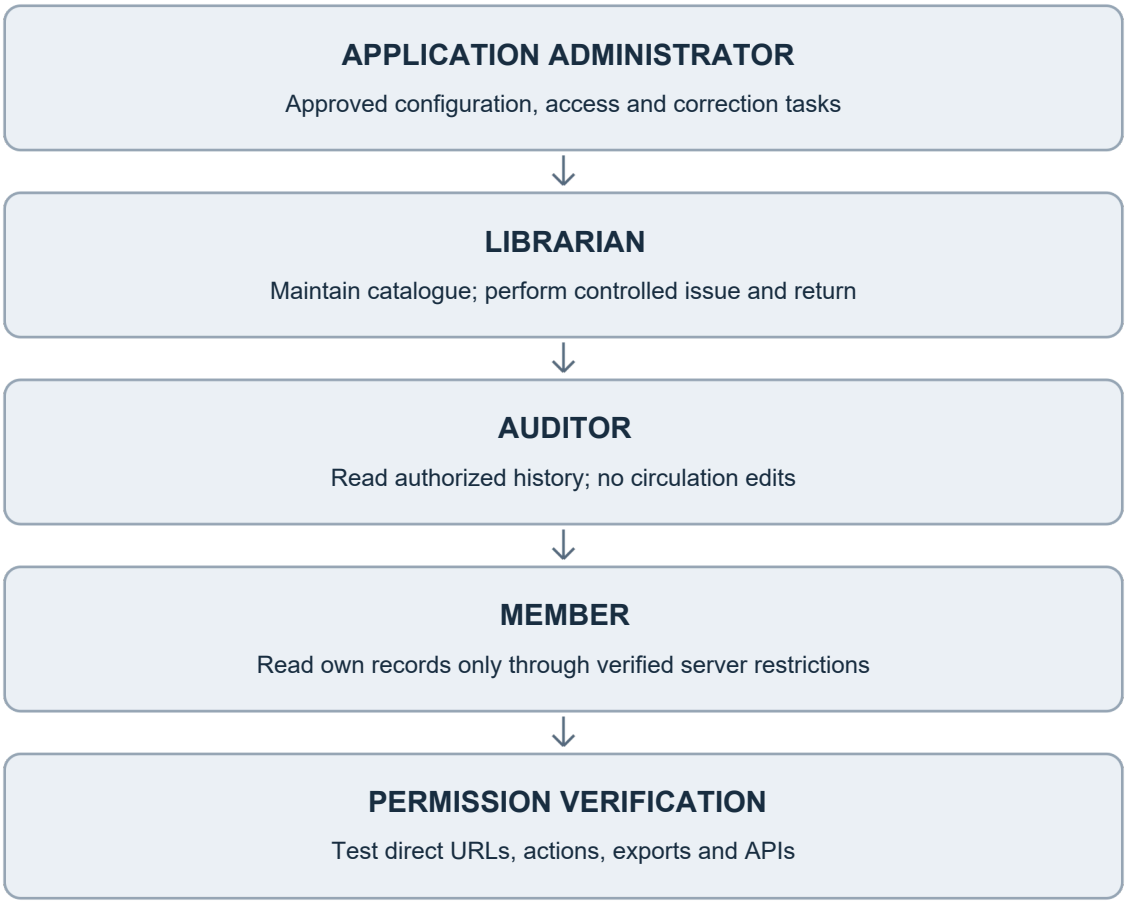
4. Change a user’s role, remove access and revoke a test integration token. Verify the effective result, including active sessions where applicable. Record who approved access and when the next review is due.

Test account	Attempt	Required result
M001 Member	Read M002 loan via copied URL or changed ID.	Denied or no record returned.
Member M001	Change Return_Date or Fine_Amount.	Denied; stored values unchanged.
Librarian	Perform a valid issue and controlled return.	Allowed with an audit entry.
Librarian	Change permission sets or app workflows. Read	Denied.
Auditor	approved history then attempt edit.	Read allowed; edit denied.
Removed user	Open private data after access revocation.	Access denied under the platform session policy.
Integration account	Use a write endpoint with read-only scope.	Denied.

Evidence

E08: assigned roles, permission sets, sharing configuration, member identity mapping, and a signed access-test matrix. Record both allowed and denied paths. Screenshots should show the test identity but redact personal details.

Use a small review register: User, Purpose, Permission Set, Role, Approver, Granted On, Review Due and Revoked On. Recheck access after a report, API or dashboard is added; a new surface can accidentally expose data even when an older surface is secure.



Generated explanatory diagram. It is not a screenshot of an executed workflow.

Component access and row access are different controls. A member who can open a report must still be prevented from reading another member's rows. Test with separate accounts, because an owner session can hide a configuration error by granting broad privileges.

Result

Expected result and verification criteria. Each role can perform only its intended tasks, member-to-member data access is blocked across all enabled surfaces, and access revocation is verified. Unverified portal scoping remains disabled.